

Line Following Robot

A Line Following Robot is an autonomous mobile robot that detects and follows a predefined path, usually a black line on a white surface or a white line on a black surface. It uses infrared (IR) sensors to continuously monitor the position of the line and adjusts the speed of its left and right motors accordingly. In this project, a PID (Proportional–Integral–Derivative) control algorithm is implemented to achieve smooth and accurate line tracking by minimizing the deviation from the path. The robot also includes a stop condition that halts movement when the line is no longer detected, making it suitable for applications such as industrial automation, warehouse transportation, and robotic competitions.

2. Software requirements

- Arduino UNO (The robot's brain)
- L298N Motor Driver (Controls the motor power)
- 5-Channel IR Sensor Array - TCRT5000 (The 5 eyes)
- 2 x DC Gear Motors (BO Motors) with Wheels
- 1 x Caster Wheel (The small front wheel)
- 12V Battery
- Robot body/chassis and some jumper wires

3. Connections:

5 IR Sensor to Arduino:

- **VCC** -> 5V
- **GND** -> GND
- **S1** -> Analog Pin A0
- **S2** -> Analog Pin A1
- **S3** -> Analog Pin A2
- **S4** -> Analog Pin A3
- **S5** -> Analog Pin A4

L298N Motor Driver to Arduino:

- **ENA** -> Pin 3
- **IN1** -> Pin 4
- **IN2** -> Pin 5
- **IN3** -> Pin 6
- **IN4** -> Pin 7
- **ENB** -> Pin 11

Battery Connections:

- **Battery (+)** -> L298N (12V input)
- **Battery (-)** -> L298N (GND) AND Arduino (GND) -> *(connect all grounds together)*

4. Understanding PID in Simple Words

PID is a smart way to keep the robot on the line. Here is what the letters mean:

- **Error:** This is how far the robot is from the line. If the line is right in the middle, the error is 0. If it drifts left or right, the error goes up (like 1, 2, 3...).
- **Kp (Proportional) = The Steering Wheel:** If the robot moves off the line, Kp pushes it back. The further away it goes, the harder Kp steers it back. You need a high Kp to make sharp turns.
- **Kd (Derivative) = The Shock Absorber:** If Kp pushes the robot back too hard, the robot will overshoot and wobble (zig-zag). Kd acts like a brake or shock absorber to smooth out the turn so the robot drives in a straight, perfectly smooth line.
- **Ki (Integral) = The Memory:** We don't need this for a line follower. We just keep it at 0.

5. Code

```
#define ENA 3
#define IN1 4
#define IN2 5
#define IN3 6
#define IN4 7
#define ENB 11

#define S1 A0
#define S2 A1
#define S3 A2
#define S4 A3
#define S5 A4

float Kp = 0;
float Ki = 0;
float Kd = 0;

int left_base_speed = 70;
```

```
int right_base_speed = 88;
```

```
int max_speed = 120;
```

```
int error = 0;
```

```
int previous_error = 0;
```

```
float P = 0, I = 0, D = 0, PID_value = 0;
```

```
bool stop_robot = false;
```

```
void setup() {
```

```
    pinMode(ENA, OUTPUT);
```

```
    pinMode(ENB, OUTPUT);
```

```
    pinMode(IN1, OUTPUT);
```

```
    pinMode(IN2, OUTPUT);
```

```
    pinMode(IN3, OUTPUT);
```

```
    pinMode(IN4, OUTPUT);
```

```
    pinMode(S1, INPUT);
```

```
    pinMode(S2, INPUT);
```

```
    pinMode(S3, INPUT);
```

```
    pinMode(S4, INPUT);
```

```
    pinMode(S5, INPUT);
```

```
    Serial.begin(9600);
```

```
}
```

```
void loop() {
```

```
    calculateError();
```

```
    if (stop_robot == true) {
```

```
        stopMotors();
```

```
} else {  
    calculatePID();  
    driveMotors();  
}  
}
```

```
void calculateError() {  
    int s1 = digitalRead(S1);  
    int s2 = digitalRead(S2);  
    int s3 = digitalRead(S3);  
    int s4 = digitalRead(S4);  
    int s5 = digitalRead(S5);
```

```
    stop_robot = false;
```

```
    if (s1 == 0 && s2 == 0 && s3 == 1 && s4 == 0 && s5 == 0)  
        error = 0;
```

```
    else if (s1 == 0 && s2 == 0 && s3 == 1 && s4 == 1 && s5 == 0)  
        error = 1;
```

```
    else if (s1 == 0 && s2 == 0 && s3 == 0 && s4 == 1 && s5 == 0)  
        error = 2;
```

```
    else if (s1 == 0 && s2 == 0 && s3 == 0 && s4 == 1 && s5 == 1)  
        error = 3;
```

```
    else if (s1 == 0 && s2 == 0 && s3 == 0 && s4 == 0 && s5 == 1)  
        error = 4;
```

```
    else if (s1 == 0 && s2 == 1 && s3 == 1 && s4 == 0 && s5 == 0)  
        error = -1;
```

```
    else if (s1 == 0 && s2 == 1 && s3 == 0 && s4 == 0 && s5 == 0)  
        error = -2;
```

```

else if (s1 == 1 && s2 == 1 && s3 == 0 && s4 == 0 && s5 == 0)
    error = -3;
else if (s1 == 1 && s2 == 0 && s3 == 0 && s4 == 0 && s5 == 0)
    error = -4;

else if (s1 == 0 && s2 == 0 && s3 == 0 && s4 == 0 && s5 == 0) {
    stop_robot = true;
}
}

```

```

void calculatePID() {
    P = error;
    I = I + error;
    D = error - previous_error;

    PID_value = (Kp * P) + (Ki * I) + (Kd * D);

    previous_error = error;
}

```

```

void driveMotors() {
    int left_speed = left_base_speed + PID_value;
    int right_speed = right_base_speed - PID_value;

    if (left_speed < 0)
        left_speed = 0;
    if (left_speed > max_speed)
        left_speed = max_speed;

    if (right_speed < 0)
        right_speed = 0;

```

```
if (right_speed > max_speed)
    right_speed = max_speed;

digitalWrite(IN1, HIGH);
digitalWrite(IN2, LOW);
digitalWrite(IN3, HIGH);
digitalWrite(IN4, LOW);

analogWrite(ENA, left_speed);
analogWrite(ENB, right_speed);
}
```

```
void stopMotors() {
    digitalWrite(IN1, LOW);
    digitalWrite(IN2, LOW);
    digitalWrite(IN3, LOW);
    digitalWrite(IN4, LOW);

    analogWrite(ENA, 0);
    analogWrite(ENB, 0);
}
```

6. Common Problems & How We Fixed Them

Problem 1: Robot doesn't drive straight

- **Why it happens:** Cheap DC motors are rarely perfectly identical. One wheel almost always spins slightly faster than the other.
- **The Fix:** We changed the `right_base_speed` in the code from 60 down to 45. This slows down the faster wheel so both wheels push equally, making the robot drive perfectly straight.

Problem 2: Robot shakes or zig-zags across the line

- **Why it happens:** The robot is trying to correct its path, but it steers too hard and overshoots the line.
- **The Fix:** We increased the Kd number to 25. This acts as a shock absorber and calms down the steering, making the ride perfectly smooth.

Problem 3: Robot gets stuck on sharp turns

- **Why it happens:** On a 90-degree turn, one wheel needs to spin backwards. If the power isn't high enough, the wheel just gets stuck and hums.
- **The Fix:** We increased the Kp number to 45. This guarantees that when a sharp turn happens, the wheel gets a huge burst of power to spin in reverse.

Problem 4: Robot loses the line completely and stops

- **Why it happens:** If the turn is too sharp, the sensors might fly off the line completely. Since all sensors see white, the robot gets confused.
- **The Fix:** We added "Smart Recovery Logic." The robot remembers its last move. If the line disappeared on the left side, the robot spins left until it finds the line again!